

FAANG-Level Frontend System Design Guide

1. Introduction

Frontend system design at FAANG level is not about building UI screens. It is about designing **scalable, performant, maintainable, and resilient systems** that can handle millions of users.

This guide gives you a structured approach to answer any frontend system design question.

2. Step-by-Step Approach (Use in Interviews)

Step 1: Clarify Requirements

Functional Requirements

- What features should the system support?
- Example:
 - User login
 - Dashboard
 - Real-time updates

Non-Functional Requirements

- Performance (load time < 2s)
 - Scalability (1M+ users)
 - Availability
 - Security
-

Step 2: High-Level Architecture

```
graph TD;
  Client[Client (Browser)] --> CDN[CDN (Static assets)];
  CDN --> Frontend[Frontend App (React/Angular)];
  Frontend --> API[API Layer (BFF)];
  API --> Backend[Backend Services (Microservices)];
```

Key Ideas: - Use CDN for static assets - Use BFF (Backend for Frontend) - Decouple frontend and backend

Step 3: Frontend Architecture

Component Architecture

- Atomic Design (Atoms → Molecules → Organisms)
- Container vs Presentational components

State Management

- Local state (UI state)
- Global state (Redux, NgRx)
- Server state (React Query)

Folder Structure Example

```
/src
  /components
  /features
  /services
  /store
  /utils
```

Step 4: Data Flow Design

- Unidirectional data flow
 - API → Store → UI
 - Avoid prop drilling (use context/store)
-

Step 5: Performance Optimization

Techniques

- Code splitting
- Lazy loading
- Tree shaking
- Memoization
- Virtual scrolling

Network Optimization

- Reduce API calls
 - Batch requests
 - Use compression (gzip/brotli)
-

Step 6: Rendering Strategy

- CSR (Client Side Rendering)

- SSR (Server Side Rendering)
- SSG (Static Site Generation)

Decision

- SEO needed → SSR
 - Static content → SSG
 - App-heavy UI → CSR
-

Step 7: Caching Strategy

- Browser caching
 - CDN caching
 - API caching
 - Service worker caching (PWA)
-

Step 8: Scalability (Micro-Frontend)

When to Use

- Large teams
- Independent deployments

Approaches

- Module Federation
 - Single-SPA
-

Step 9: Security

- XSS protection
 - CSRF protection
 - Token-based authentication (JWT)
 - Secure storage (avoid localStorage for sensitive data)
-

Step 10: Error Handling & Resilience

- Global error handling
 - Retry logic
 - Fallback UI
 - Graceful degradation
-

Step 11: Testing Strategy

- Unit testing

- Integration testing
 - E2E testing
-

Step 12: Monitoring & Observability

- Logging
 - Error tracking (Sentry)
 - Performance monitoring
-

3. Real Example: Design a Dashboard System

Requirements

- Display charts
- Real-time updates
- Filters

Solution

Architecture

- React frontend
- BFF layer
- WebSocket for real-time data

Optimizations

- Lazy load charts
 - Virtual scroll for large lists
 - Cache API responses
-

4. Trade-offs (Very Important for FAANG)

Decision	Pros	Cons
CSR	Fast interactions	Poor SEO
SSR	Good SEO	Server cost
Micro-frontend	Scalable teams	Complex

5. Common Mistakes

- Over-engineering early
- Ignoring performance
- Tight coupling

- No caching strategy
-

6. FAANG-Level Answer Template

Use this structure in interviews:

1. Clarify requirements
 2. Define architecture
 3. Design components & state
 4. Optimize performance
 5. Choose rendering strategy
 6. Add caching
 7. Ensure security
 8. Handle errors
 9. Add testing & monitoring
-

7. Final Tips

- Always talk about trade-offs
 - Think in scale (millions of users)
 - Focus on user experience
 - Keep system modular
-

This guide is enough to answer almost any frontend system design question at FAANG level.